

Advanced NumPy

Read the bullets, predict what the code will print, then check yourself against the output shown below each block.

- **Covered here:** broadcasting, reshaping, stacking, conditional logic, linear algebra, copy vs view
- **Why these:** they let you reorganise, combine and transform whole tables of numbers without a single loop
- **How each section is built:** what the operation does -> how to think about it -> code -> output explained -> what usually goes wrong

```
import numpy as np
print(np.__version__)
```

2.4.4

1. Broadcasting

The problem it solves

- Normal arithmetic works position by position, so both arrays must have the **same shape**
- Real data almost never matches: 100 students x 3 subjects, but only **3** bonus values to add
- Looping over this would be slow and clumsy
- **Broadcasting** = when shapes differ, NumPy "stretches" the smaller array to fit the bigger one
- The stretching is **virtual** - no copies are made in memory, which is why it stays fast
- You have already used it: in `arr + 5`, the single number 5 is stretched across every element

1.1 A 2D matrix with a 1D row

- The most common case: matrix (3, 3) combined with a row (3,)
- The row is applied to **every row** of the matrix
- Result: each **column** ends up with its own value

matrix (3,3)		row (3,)		NumPy pretends the row is:
<code>[[1 1 1]</code>				<code>[[10 20 30]</code>
<code>[1 1 1]</code>	<code>+</code>	<code>[10 20 30]</code>	<code>-></code>	<code>[10 20 30]</code>
<code>[1 1 1]]</code>				<code>[10 20 30]]</code>

```
m = np.ones((3, 3))          # 3x3 matrix of ones
row = np.array([10, 20, 30]) # shape (3,)

print("matrix shape:", m.shape)
print("row shape   :", row.shape)
print(m + row)
```

```
matrix shape: (3, 3)
row shape   : (3,)
[[11. 21. 31.]
 [11. 21. 31.]
 [11. 21. 31.]]
```

- Column 0 received 10, column 1 received 20, column 2 received 30
- Use this pattern for: subject-wise bonus, per-feature scaling, subtracting a per-column average

1.2 A column vector with a row vector

- Column (3, 1) meets row (3,) - **neither** fits the other
- So NumPy stretches **both**: the column across 3 columns, the row down 3 rows

- Result is a full 3 x 3 grid - you get **9** numbers, not 3
- Element `[i][j] = col[i] + row[j]` - every possible combination

```
col = np.array([[1], [2], [3]]) # shape (3, 1)
row = np.array([10, 20, 30])    # shape (3,)

result = col + row
print("result shape:", result.shape)
print(result)
```

```
result shape: (3, 3)
[[11 21 31]
 [12 22 32]
 [13 23 33]]
```

- **Useful for:** multiplication tables, distance grids, outer products
- **Dangerous because:** passing a column where a row was expected gives a grid, not an error
- The wrong answer then flows silently into the rest of your code

WATCH OUT: The lesson is not "avoid broadcasting" - it is **always check the shape of the result**. If you expected 3 numbers and `.shape` says `(3, 3)`, you have found your bug.

1.3 The broadcasting rule

- Line up the two shapes **from the right**, compare dimension by dimension
- Two dimensions are compatible when they are **equal**, or when **one of them is 1**
- The dimension that is 1 gets stretched
- A missing dimension on the left counts as 1 - that is why `(3, )` works with `(3, 3)`
- If any pair fails both tests, NumPy raises an error

```
(3, 3) + (3,)    -> ok      (3,) is read as (1, 3); the 1 stretches to 3
(3, 1) + (3,)    -> ok      both stretch, result is (3, 3)
(4, 3) + (3,)    -> ok      the row applies to all 4 rows
(4, 3) + (4,)    -> ERROR   compare from the right: 3 vs 4, neither equal nor 1
(3,)   + (4,)    -> ERROR   3 and 4 are neither equal nor 1
```

- Note line 4: `(4, 3)` with `(4, )` **fails**, even though 4 appears in both shapes
- Reason: comparison starts from the right, so 3 is compared against 4
- To apply one value per **row**, reshape it into a column first: `(4, 1)`

```
a = np.array([1, 2, 3])
b = np.array([1, 2, 3, 4])

print("a:", a.shape, " b:", b.shape)

# Uncomment the next line to see the failure for yourself:
# a + b
#
# ValueError: operands could not be broadcast together with shapes (3,) (4,)
```

```
a: (3,) b: (4,)
```

- Never skim past this error - NumPy prints **both shapes** in the message
- That tells you exactly which array is wrong and what it needs to become
- Most shape bugs are solved by reading the error, not by guessing

1.4 Practical use: column-wise normalization

- Problem: height is around 170, weight is around 60 - completely different scales

- Comparing or combining such columns is unfair: the larger-valued column dominates purely because of its units
- **Standardization fix:** subtract each column's mean, divide by each column's std
- Every column then centres around 0 with a comparable spread
- Key insight: `X.mean(axis=0)` gives **one value per column** - shape `(2,)`, exactly what broadcasting needs

```
X = np.array([[170, 65],
              [155, 50],
              [180, 80],
              [165, 58]])

col_mean = X.mean(axis=0)      # one mean per column
col_std  = X.std(axis=0)       # one std per column

print("col_mean:", col_mean, "shape", col_mean.shape)
X_scaled = (X - col_mean) / col_std  # (4,2) - (2,) -> broadcasting
print(np.round(X_scaled, 2))
```

```
col_mean: [167.5  63.25] shape (2,)
[[ 0.28  0.16]
 [-1.39 -1.2 ]
 [ 1.39  1.52]
 [-0.28 -0.48]]
```

- One line, no loop, works the same for 4 rows or 4 million
- Scaled values now sit roughly between -1.5 and 1.5 for **both** columns

NOTE: Why `axis=0` is correct here: **the axis you pass is the one that collapses**. `axis=0` collapses the rows and leaves one answer per column. `axis=1` would give one answer per row - not what column-wise scaling needs.

2. Reshaping and Dimensions

What reshaping actually is

- Reshaping changes how data is **organised**, never what the data *is*
- Values stay the same, their order stays the same - only the row/column arrangement changes
- Mental picture: 12 chairs can be 3 rows of 4 or 4 rows of 3 - still the same 12 chairs
- Why it matters: ML libraries are strict about shapes and will refuse a 1D array even when the numbers are correct

2.1 reshape(rows, columns)

- **Golden rule:** rows x columns must equal the total number of elements
- With 12 elements you can ask for `(3, 4)`, `(4, 3)`, `(2, 6)`, `(6, 2)`, `(12, 1)`, `(1, 12)` - any product of 12
- Filling order is **row by row**, left to right
- That is why `arange(1, 13).reshape(3, 4)` starts its second row at 5, not at 2

```
arr = np.arange(1, 13)          # 12 elements
print(arr, "| size:", arr.size)

print("\n3 x 4:")
print(arr.reshape(3, 4))

print("\n4 x 3:")
print(arr.reshape(4, 3))
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12] | size: 12

3 x 4:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

4 x 3:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

- Now the failure case: (3, 5) asks for 15 slots, but only 12 values exist

```
print("size available:", arr.size, " slots requested:", 3 * 5)

# Uncomment the next line to see the failure for yourself:
# arr.reshape(3, 5)
#
# ValueError: cannot reshape array of size 12 into shape (3,5)
```

```
size available: 12  slots requested: 15
```

WATCH OUT: reshape does **not** modify the array in place - it returns a **new** array. `arr.reshape(3, 4)` on its own line does nothing; you must assign it: `m = arr.reshape(3, 4)`. Nothing errors out, so the bug stays silent.

2.2 The -1 trick

- -1 means: *"NumPy, you work this dimension out for me"*
- NumPy divides the total size by the dimension you did specify
- With 12 elements: `reshape(3, -1) -> (3, 4)`, `reshape(-1, 2) -> (6, 2)`
- **Only one dimension can be -1** - two unknowns and one equation is an error
- Good practice, not laziness: if your data grows from 100 rows to 10,000, the code still works unchanged

```
arr = np.arange(1, 13)

print(arr.reshape(3, -1).shape)  # NumPy computes 4
print(arr.reshape(-1, 2).shape)  # NumPy computes 6
print(arr.reshape(-1, 1).shape)  # single column
```

```
(3, 4)
(6, 2)
(12, 1)
```

2.3 reshape(-1, 1): turning a list into a column

- The most-used reshape of all, so it deserves its own section
- A 1D array (n,) is just a sequence - it has **no notion of rows and columns**
- Many operations (joining tables side by side, matrix multiplication) need a real **2D column**

- `reshape(-1, 1)` converts `(n,)` into `(n, 1)`: `n` rows, exactly one column
- You will need it again in section 3.2, where joining two 1D arrays fails without it

```
marks = np.array([65, 70, 88, 92])
print("1D shape:", marks.shape)      # (4,) -> a flat sequence
print(marks)

col = marks.reshape(-1, 1)
print("\n2D shape:", col.shape)      # (4, 1) -> a real column
print(col)
```

```
1D shape: (4,)
[65 70 88 92]

2D shape: (4, 1)
[[65]
 [70]
 [88]
 [92]]
```

- The brackets are the visual signature of the shape:
- `[65 70 88 92]` -> one flat sequence
- `[[65], [70], [88], [92]]` -> four rows, one value each
- Same numbers, completely different structure

NOTE: Many libraries refuse a 1D array and ask for a 2D one instead. Whenever you see a message containing `"Expected 2D array, got 1D array instead"`, `reshape(-1, 1)` is the fix.

2.4 `flatten()` vs `ravel()`

- Both flatten any array into 1D; values and order are identical
- The difference only appears when you **modify** the result:
- `flatten()` -> returns a **copy** (own memory, original always safe, costs time/memory)
- `ravel()` -> returns a **view** where possible (shares memory, changes reach the original, faster)
- Predict the output below before reading it

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])

f = m.flatten()
r = m.ravel()
print("flatten:", f)
print("ravel  :", r)

f[0] = 999          # modifying the copy
r[1] = 888          # modifying the view
print("\noriginal matrix after both edits:")
print(m)
```

```
flatten: [1 2 3 4 5 6]
ravel  : [1 2 3 4 5 6]

original matrix after both edits:
[[ 1 888  3]
 [ 4  5  6]]
```

- The 999 never reached the original; the 888 did
- **Default choice:** `flatten()` - the small cost buys protection from accidental edits
- Use `ravel()` only for large arrays when you are certain you will not modify the result

2.5 `np.newaxis`

- Inserts a **new dimension of size 1** wherever you place it
- Does the same job as reshape, but reads more clearly - it says "add an axis here"
- `[:, np.newaxis]` -> axis added after -> **column**
- `[np.newaxis, :]` -> axis added in front -> **row**

```
marks = np.array([65, 70, 88, 92])

as_column = marks[:, np.newaxis]    # same as reshape(-1, 1)
as_row    = marks[np.newaxis, :]    # same as reshape(1, -1)

print("original :", marks.shape)
print("as_column:", as_column.shape)
print("as_row    :", as_row.shape)
print(as_row)
```

```
original : (4,)
as_column: (4, 1)
as_row    : (1, 4)
[[65 70 88 92]]
```

NOTE: Three shapes to keep straight forever: (4,) is 1D with no row/column structure, (4, 1) is a column of 4 rows, (1, 4) is a single row of 4 columns. Same four numbers, different behaviour in broadcasting, matrix multiplication and ML libraries.

3. Stacking and Combining Arrays

Why stacking matters

- Data rarely arrives in one piece: heights from one source, weights from another
- Last month's records in one array, this month's in another
- Stacking assembles the pieces into one array you can actually work with
- **Mental picture:** a new record is a new **row**, a new measurement is a new **column**

3.1 vstack and hstack

- `np.vstack` -> stacks **vertically**, rows grow; the arrays must have the same number of **columns**
- `np.hstack` -> stacks **horizontally**, columns grow; the arrays must have the same number of **rows**
- The names are literal: v = vertical, h = horizontal
- Whichever direction you stack in, the **other** dimension must already match

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = np.array([[7, 8, 9]])

print("vstack:")
print(np.vstack((A, B)), "-> shape", np.vstack((A, B)).shape)

C = np.array([[10],
              [20]])
print("\nhstack:")
print(np.hstack((A, C)), "-> shape", np.hstack((A, C)).shape)
```

```
vstack:
[[1 2 3]
 [4 5 6]
 [7 8 9]] -> shape (3, 3)

hstack:
[[ 1  2  3 10]
 [ 4  5  6 20]] -> shape (2, 4)
```

- vstack: (2, 3) + (1, 3) -> (3, 3) - rows added up, columns stayed at 3
- hstack: (2, 3) + (2, 1) -> (2, 4) - columns added up, rows stayed at 2

WATCH OUT: The arrays go in as a **tuple** - two sets of brackets: `np.vstack((A, B))`. `np.vstack(A, B)` raises a `TypeError`. Same pattern as `np.zeros((3, 4))`: one argument that happens to be a tuple, not two arguments.

3.2 The 1D hstack surprise

- You have two 1D arrays and want a two-column dataset - hstack sounds right
- But 1D arrays have **no columns to stack**
- NumPy just joins them end to end and returns one long 1D array
- No error, no warning - the wrong structure, silently
- **Fix:** give each array a column shape first with `reshape(-1, 1)`, then stack

```
heights = np.array([170, 155, 180, 165])
weights = np.array([65, 50, 80, 58])

wrong = np.hstack((heights, weights))
print("wrong shape:", wrong.shape)
print(wrong)

X = np.hstack((heights.reshape(-1, 1), weights.reshape(-1, 1)))
print("\ncorrect shape:", X.shape)
print(X)
```

```
wrong shape: (8,)
[170 155 180 165  65  50  80  58]

correct shape: (4, 2)
[[170  65]
 [155  50]
 [180  80]
 [165  58]]
```

- (8,) = eight numbers in a line
- (4, 2) = four records with two measurements each
- Only the second one is a usable table

3.3 np.concatenate

- The general-purpose version of both functions above

- You choose the direction with `axis` instead of a different function name
- `axis=0` -> joins along rows (same as `vstack`)
- `axis=1` -> joins along columns (same as `hstack`)
- Same `axis` idea you used with `mean` and `sum`
- **Use `vstack/hstack` for readable code; use `concatenate` when the direction comes from a variable or you work beyond 2D**

```
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

print("axis=0:\n", np.concatenate((A, B), axis=0))
print("\naxis=1:\n", np.concatenate((A, B), axis=1))
```

```
axis=0:
[[1 2]
 [3 4]
 [5 6]
 [7 8]]

axis=1:
[[1 2 5 6]
 [3 4 7 8]]
```

3.4 Practical use: adding a computed column

- Common task: you have a table and want to attach a **new column you calculated yourself**
- Example: a marks table for 3 subjects, and you want the total as a 4th column
- Step 1 - compute the totals with `sum(axis=1)`, which gives one value per row
- Step 2 - that result is 1D, so `reshape(-1, 1)` turns it into a column
- Step 3 - `hstack` attaches it to the right of the original table

```
marks = np.array([[70, 65, 80],
                  [55, 90, 72],
                  [88, 76, 91]])

totals = marks.sum(axis=1) # one total per student -> shape (3,)
print("totals:", totals, "shape", totals.shape)

table = np.hstack((marks, totals.reshape(-1, 1)))
print("\nbefore:", marks.shape, " after:", table.shape)
print(table)
```

```
totals: [215 217 255] shape (3,)

before: (3, 3) after: (3, 4)
[[ 70  65  80 215]
 [ 55  90  72 217]
 [ 88  76  91 255]]
```

- The table went from 3 columns to 4, with the last column holding each student's total
- Note the two ideas working together: `axis=1` to collapse the columns, `reshape(-1, 1)` to make the result stackable

4. Conditional Logic with `np.where`

From if-else to vectorized decisions

- Plain Python needs a loop: check each element, decide, append

- NumPy replaces the whole pattern with one function
- `np.where(condition, value_if_true, value_if_false)`
- Evaluates the condition for **every** element at once and picks the matching value
- Read it as: *what is the test / what do I want if it passes / what do I want if it fails*

4.1 The basic form

- Step 1: `marks >= 40` produces a boolean array of True and False
- Step 2: `np.where` walks that boolean array and picks "Pass" for True, "Fail" for False
- Output shape always matches the input - one decision per element

```
marks = np.array([35, 68, 42, 91, 27, 55])

result = np.where(marks >= 40, "Pass", "Fail")
print(marks)
print(result)
```

```
[35 68 42 91 27 55]
['Fail' 'Pass' 'Pass' 'Pass' 'Fail' 'Pass']
```

- Line the two printed arrays up position by position to verify: 35 failed, 68 passed, 42 passed

4.2 Cleaning data with np.where

- The most common real use is **repairing** data, not labelling it
- Sensors record impossible negatives; a score reads 120 out of 100; an age column holds 999
- Pattern: `np.where(condition, replacement, arr)`
- The **third** argument is the original array - meaning "if false, keep what was already there"

```
arr = np.array([3, -7, 2, -1, 9, -4])
print("negatives to zero :", np.where(arr < 0, 0, arr))

scores = np.array([88, 105, 96, 120, 74])
print("capped at 100      :", np.where(scores > 100, 100, scores))
```

```
negatives to zero : [3 0 2 0 9 0]
capped at 100      : [ 88 100  96 100  74]
```

- Only the three negatives changed in the first line
- Only the two values above 100 were pulled down in the second
- Everything else passed through untouched - exactly what cleaning should do

4.3 Nested np.where

- `np.where` handles two outcomes; for three or more, put another `np.where` in the "false" slot
- Meaning: **"if the first test failed, here is the next test to try"**
- **Order is critical:** check the highest threshold first
- If you tested `>= 50` before `>= 75`, every strong student would be caught by 50 and nobody would get an A

```
marks = np.array([92, 61, 45, 78, 33, 55])

grades = np.where(marks >= 75, 'A',
                  np.where(marks >= 50, 'B', 'C'))
print(marks)
print(grades)
```

```
[92 61 45 78 33 55]
['A' 'B' 'C' 'A' 'C' 'B']
```

- Trace 61: fails ≥ 75 -> moves to the inner where -> passes ≥ 50 -> gets B
- The second condition never needs to say "between 50 and 74" - anything reaching it already failed 75

WATCH OUT: Keep both outcomes the **same type**. `np.where(cond, "Pass", 0)` turns everything into strings, because an array holds only one dtype. You get text '0', and later arithmetic fails.

- Nested `np.where` becomes unreadable beyond three or four levels
- At that point, look up `np.select` - it takes a list of conditions and a list of results

4.4 np.where with only a condition

- Called with just a condition, the behaviour changes completely
- It returns the **positions** where the condition is true, not chosen values
- The return is a **tuple**, not an array - for 1D input, write `idx[0]` to reach the indices
- The tuple exists because 2D input needs one array of rows and one of columns

```
marks = np.array([35, 68, 42, 91, 27, 55])

idx = np.where(marks > 50)
print("raw return:", idx)           # a tuple
print("indices  :", idx[0])
print("values   :", marks[idx])
```

```
raw return: (array([1, 3, 5]),)
indices    : [1 3 5]
values     : [68 91 55]
```

- Feeding the indices back into the array retrieves the matching values
- Advantage over plain boolean filtering: you also learn **where** those values were

```
# On a 2D array you get one array of row indices and one of column indices
m = np.array([[10, 80],
              [55, 20]])

rows, cols = np.where(m > 50)
print("rows:", rows, "cols:", cols)
print("values:", m[rows, cols])
```

```
rows: [0 1] cols: [1 0]
values: [80 55]
```

- Read the output as coordinates: (0, 1) and (1, 0)
- (0, 1) -> row 0, column 1 -> the value 80
- (1, 0) -> row 1, column 0 -> the value 55

5. Linear Algebra Operations

Why this section exists

- Multiplying whole tables at once is what makes NumPy more than a container for numbers
- These operations answer questions like "what is each student's weighted total?" in a single line
- They are also the foundation of almost every numerical technique you will meet later

5.1 Element-wise multiplication vs the dot product

- These look almost identical in code and mean entirely different things
- `a * b` -> multiplies position by position -> result is an **array** of the same size
- `np.dot(a, b)` -> multiplies position by position **and adds it all up** -> result is a single **number**
- `[1,2,3] * [4,5,6]` -> `[4, 10, 18]`

- `np.dot([1,2,3], [4,5,6]) -> 4 + 10 + 18 = 32`
- The multiplication step is identical; the dot product just does not stop there
- **Meaning of a dot product:** a weighted total (marks x subject weights = final score) - used again in section 5.4

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print("a * b      :", a * b)          # element-wise
print("np.dot     :", np.dot(a, b))   # 1*4 + 2*5 + 3*6
print("a @ b      :", a @ b)          # same thing, modern syntax
```

```
a * b      : [ 4 10 18]
np.dot     : 32
a @ b      : 32
```

- `@` does exactly what `np.dot` does but reads better in long expressions - prefer it in new code

5.2 Matrix multiplication

- For 2D arrays, `@` performs true matrix multiplication
- Rule for each output element: take a **row** from the first matrix, a **column** from the second, dot them
- Element `[0][0]` = row 0 with column 0; element `[0][1]` = row 0 with column 1

```
(m x n) @ (n x p) = (m x p)
  ^         ^
  |         |
  |_____|
  these must match ("the handshake")
```

- The **inner** numbers must be equal - a row and a column must be the same length to be dotted
- The **outer** numbers give the shape of the result
- Habit worth building: write the shapes down before multiplying - five seconds, prevents most errors

```
A = np.array([[1, 2, 3],
               [4, 5, 6]])          # (2, 3)
B = np.array([[7, 8],
               [9, 10],
               [11, 12]])          # (3, 2)

print("A:", A.shape, " B:", B.shape)
print("A @ B ->", (A @ B).shape)
print(A @ B)
```

```
A: (2, 3) B: (3, 2)
A @ B -> (2, 2)
[[ 58  64]
 [139 154]]
```

- Verify the top-left value by hand: row 0 of A with column 0 of B
- $1*7 + 2*9 + 3*11 = 7 + 18 + 33 = 58$
- Next, the failure case: both matrices below are (2, 3), so the inner dimensions are 3 and 2

```
C = np.array([[1, 2, 3],
              [4, 5, 6]])          # (2, 3)

print("A:", A.shape, " C:", C.shape, " -> inner dimensions 3 and 2 do not match")

# Uncomment the next line to see the failure for yourself:
# A @ C
#
# ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0 ...
#         (size 2 is different from 3)
```

```
A: (2, 3) C: (2, 3) -> inner dimensions 3 and 2 do not match
```

- The message is dense, but the useful part is at the end: one size is 2, the other is 3
- That is your inner-dimension mismatch - the fix is almost always a transpose

5.3 Transpose

- `.T` flips the array over its diagonal: rows become columns, columns become rows
- `(2, 3)` becomes `(3, 2)`; nothing is lost, the values are just re-labelled
- In practice it is your **shape-fixing tool** when two matrices refuse to multiply

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

print("A shape  :", A.shape)
print("A.T shape:", A.T.shape)
print(A.T)

print("\nA @ A.T works:", (A @ A.T).shape)
```

```
A shape  : (2, 3)
A.T shape: (3, 2)
[[1 4]
 [2 5]
 [3 6]]

A @ A.T works: (2, 2)
```

- `A @ A.T` is `(2, 3) @ (3, 2)` -> inner dimensions 3 and 3 -> handshake succeeds -> result `(2, 2)`
- Transposing turned an impossible multiplication into a valid one

WATCH OUT: Transposing a **1D** array does nothing. `np.array([1, 2, 3]).T` still has shape `(3,)` - transpose swaps two dimensions, and 1D has only one. For a real column, use `reshape(-1, 1)`.

```
v = np.array([1, 2, 3])
print("v.T shape      :", v.T.shape)          # unchanged
print("reshape(-1,1)  :", v.reshape(-1, 1).shape)
```

```
v.T shape      : (3,)
reshape(-1,1)  : (3, 1)
```

5.4 Putting it together: weighted totals for a whole table

- Scenario: 4 students, 3 subjects, and each subject carries a different weight in the final result
- For **one** student: `mark1*w1 + mark2*w2 + mark3*w3` - a dot product of that row with the weights
- For **every** student at once: `final = M @ w` - one operation, all rows
- Identical code whether the table has 4 rows or 4 million
- Shapes: `(4, 3) @ (3,)` -> `(4,)`, which is one number per student

```
M = np.array([[70, 65, 80],      # marks: 4 students x 3 subjects
              [55, 90, 72],
              [88, 76, 91],
              [60, 58, 66]])
w = np.array([0.3, 0.3, 0.4])    # subject weights, adding up to 1

final = M @ w
print("M:", M.shape, "w:", w.shape, "-> final:", final.shape)
print(np.round(final, 2))
```

```
M: (4, 3) w: (3,) -> final: (4,)
[72.5 72.3 85.6 61.8]
```

- Check the first student by hand: $70 \times 0.3 + 65 \times 0.3 + 80 \times 0.4 = 21 + 19.5 + 32 = 72.5$
- One line replaced a nested loop over students and subjects
- Change the weights and the whole table is re-scored instantly

NOTE: A (4, 3) matrix times a (3,) vector gives (4,) - one result per row. Whenever a result surprises you, print the input shapes before debugging anything else.

5.5 Order matters

- With ordinary numbers $3 \times 5 = 5 \times 3$; with matrices this is **not** true
- Matrix multiplication is **not commutative**: $A @ B$ and $B @ A$ generally differ
- For non-square matrices, one of the two is usually not even legal
- Practical consequence: the order in any matrix formula is fixed - swapping two matrices gives an error or a quietly wrong result

```
np.random.seed(0)
A = np.random.randint(1, 5, (2, 2))
B = np.random.randint(1, 5, (2, 2))

print("A @ B:\n", A @ B)
print("B @ A:\n", B @ A)
print("equal?", np.array_equal(A @ B, B @ A))
```

```
A @ B:
[[20 20]
 [12 12]]
B @ A:
[[12 20]
 [12 20]]
equal? False
```

6. Copy vs View

The rule that catches everyone once

- Slicing a Python **list** gives a new independent list
- Slicing a NumPy **array** does not - you get a **view**, a window onto the original memory
- Both names point to the same data, so editing the slice edits the original
- **Why:** copying large arrays constantly would be wasteful, so slicing is designed to be free
- The cost of that speed is this behaviour - better to meet it here than in a real project

```
arr = np.array([10, 20, 30, 40, 50])

s = arr[1:4]          # a view
s[0] = 999
print("slice   :", s)
print("original:", arr)    # the original changed too!
```

```
slice   : [999  30  40]
original: [ 10 999  30  40  50]
```

- The 20 became 999 without ever being touched directly
- `s[0]` and `arr[1]` are the same memory location under two names
- **Fix:** call `.copy()` when you want independence

```
arr = np.array([10, 20, 30, 40, 50])

s = arr[1:4].copy()   # an independent copy
s[0] = 999
print("copy    :", s)
print("original:", arr)    # untouched
```

```
copy    : [999  30  40]
original: [10 20 30 40 50]
```

NOTE: Remember it as: **a slice is a window**, **.copy() is a photograph**. Change what you see through a window and the real thing changes; draw on a photograph and the original is safe. Same rule as `ravel()` (view) vs `flatten()` (copy) from section 2.4.

- **Practical habit:** before modifying a slice - cleaning, capping, replacing - make a copy first
- Skip the copy only when you genuinely intend to change the original

7. Quick Reference

- Use this table for revision
- If a row does not make immediate sense, go back and re-read that section

Operation	What it does	Watch out for
<code>a + b</code> (different shapes)	Broadcasting: stretches the smaller shape to fit	Compare shapes from the right; 3 vs 4 fails
<code>X - X.mean(axis=0)</code>	Subtracts from each column its own mean	<code>axis=0</code> collapses rows, giving one value per column
<code>reshape(r, c)</code>	Reorganises the same data into a new shape	<code>r x c</code> must equal size; returns a new array, so assign it
<code>reshape(-1, 1)</code>	Turns a 1D sequence into a real 2D column	Only one dimension can be -1
<code>flatten()</code> / <code>ravel()</code>	Converts any array to 1D	<code>flatten</code> is a copy (safe), <code>ravel</code> is a view (fast)
<code>np.newaxis</code>	Adds a dimension of size 1	Position decides whether you get a row or a column
<code>vstack</code> / <code>hstack</code>	Joins arrays by rows / by columns	Pass a tuple; 1D <code>hstack</code> just joins end to end
<code>concatenate(axis=)</code>	General joining in any direction	<code>axis=0</code> is vertical, <code>axis=1</code> is horizontal
<code>np.where(c, x, y)</code>	Vectorized if-else over a whole array	Mixing types converts everything to strings
<code>np.where(c)</code>	Returns the positions where <code>c</code> is true	Returns a tuple; index it with <code>[0]</code> for 1D

Operation	What it does	Watch out for
<code>a * b</code> vs <code>a @ b</code>	Element-wise product vs dot / matrix product	Completely different operations, similar syntax
<code>A @ B</code>	Matrix multiplication	$(m \times n) @ (n \times p)$: the inner dimensions must match
<code>.T</code>	Transpose - swaps rows and columns	Does nothing to a 1D array
<code>.copy()</code>	Creates an independent duplicate	Slices are views by default and share memory

What to do next

- Re-read this notebook with the outputs cleared and predict each result before revealing it
- Wrong predictions mark exactly where your understanding needs another pass
- Then attempt the assignment **without** looking back at these notes
- Struggling first and checking afterwards teaches far more than reading alone